

Project: Infrastructure as Code with Terraform or CloudFormation

Services: VPC, EC2, RDS, S3, IAM (via Terraform or CFN)

Goal: Build a complete VPC + 3-tier architecture using IaC tools.

Skills: Reusability, version control, modularization.

Definition:

This project focuses on designing and deploying a secure and scalable 3-tier cloud architecture on Amazon Web Services using Infrastructure as Code (IaC) with Terraform. The infrastructure is created inside a Amazon VPC, which provides an isolated network environment with public and private subnets to separate different application layers. The Web Tier uses Amazon EC2 instances in a public subnet to handle user requests, while the Application Tier processes the application logic within a private subnet for improved security. The Database Tier stores and manages application data using Amazon RDS, ensuring reliable and secure database management. Additionally, Amazon S3 is used for object storage, and access permissions are controlled through AWS Identity and Access Management. By using Terraform, the entire infrastructure is automated, repeatable, and easy to manage, demonstrating modern DevOps practices for efficient cloud resource provisioning and scalable application deployment.

Scope of the Project:

- Design and deploy a 3-tier cloud architecture on Amazon Web Services.
- Use Infrastructure as Code (IaC) with Terraform to automate the provisioning of cloud resources.
- Create a secure network environment using VPC with properly configured public and private subnets.
- Deploy web servers in the Web Tier using Amazon EC2 to handle user requests and deliver application content.
- Implement a Database Tier using Amazon RDS for secure and reliable data storage.
- Use Amazon S3 for storing application files, backups, or static content.
- Configure secure access and permissions using AWS Identity and Access Management.
- Ensure scalability, security, and efficient management of infrastructure through automated deployment.
- Demonstrate modern DevOps practices such as automation, repeatability, version-controlled infrastructure.

Methodology:

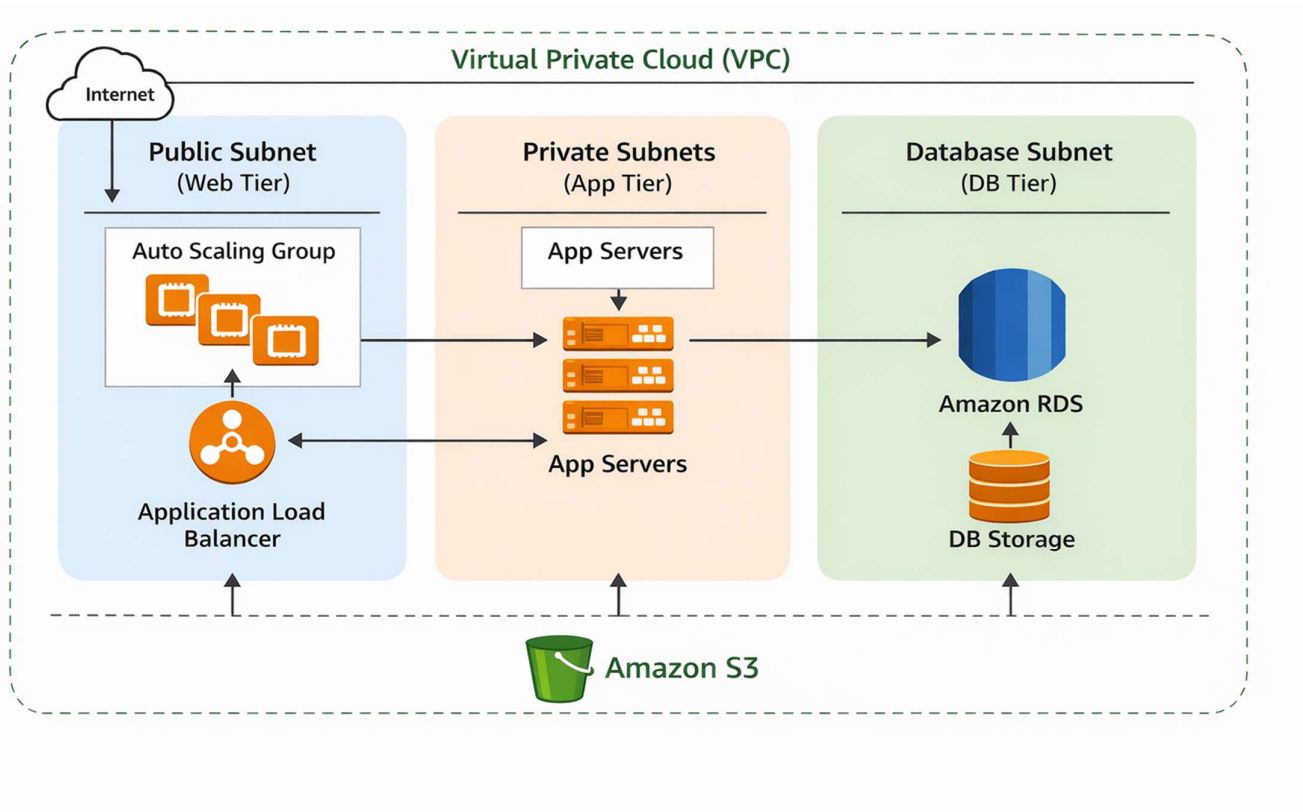
1.Terraform (Infrastructure as Code – Automated Method)

- Uses Terraform configuration files to automate infrastructure provisioning on Amazon Web Services.
- Automates the creation of VPC, subnets, EC2 instances, load balancer, and RDS database.
- Ensures consistent and repeatable infrastructure deployment using code.
- Reduces manual configuration errors and simplifies infrastructure management.
- Suitable for scalable environments, DevOps practices, and automated cloud deployments.

2.AWS Management Console (Manual Configuration Method)

- Uses services such as Amazon VPC, Amazon EC2, AWS Elastic Load Balancing, and Amazon RDS.
- Allows manual creation of networking components like public and private subnets inside a VPC
- Enables deployment of EC2 instances and configuration of security groups through the AWS console.
- Helps understand the architecture and working of cloud resources step-by-step.
- Suitable for learning purposes, testing, and small academic projects.

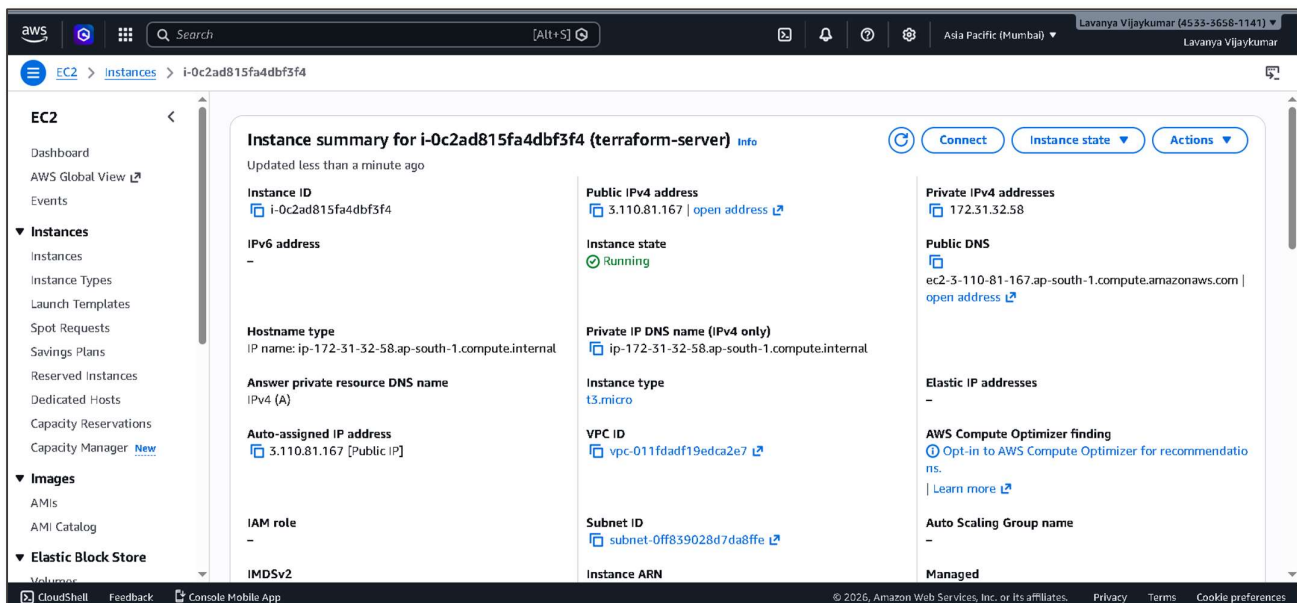
Architecture Diagram:



Project Implementation Step

STEP 1: Create EC2 Instance

- Click Launch Instance.
- Name: Terraform-Server.
- Select AMI: Ubuntu Server 22.04 LTS.
- Choose Instance Type: t3.micro
- Click Launch Instance to create the server.



STEP 2: Install Terraform

- Install Terraform
- Check with Version = terraform -version

```

aws  [Search] [Alt+S] [Adia Pacific (Mumbai)] Lavanya Vijaykumar (4533-3658-1141) Lavanya Vijaykumar

System information as of Sat Mar 7 06:28:16 UTC 2026

System load: 0.0      Processes:    106
Usage of /:  48.6% of 7.57GB   Users logged in: 1
Memory usage: 24%      IPv4 address for ens5: 172.31.32.58
Swap usage:  0%

* Ubuntu Pro delivers the most comprehensive open source security and
  compliance features.
  https://ubuntu.com/aws/pro

Expanded Security Maintenance for Applications is not enabled.

65 updates can be applied immediately.
57 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Sat Mar 7 04:01:01 2026 from 13.233.177.5
ubuntu@ip-172-31-32-58:~$ terraform -version
Terraform v1.7.5
on linux_amd64
  
```

STEP 3: Configure AWS Credentials

- Configure access to Amazon Web Services using AWS CLI.
- Command: aws configure
- Enter:
- Access Key
- Secret Key
- Region
- Output format

```

ubuntu@ip-172-31-32-58:~$ aws configure
AWS Access Key ID [*****4GFK]:
AWS Secret Access Key [*****nWjC]:
Default region name [ap-south-1]:
Default output format [json]:
ubuntu@ip-172-31-32-58:~$ aws sts get-caller-identity
{
  "UserId": "453336581141",
  "Account": "453336581141",
  "Arn": "arn:aws:iam::453336581141:root"
}
ubuntu@ip-172-31-32-58:~$
  
```

STEP 4: Create Terraform Configuration Files

- First create an directory inside an directory create this files.
- main.tf – defines AWS resources

```

aws
Search [Alt+S] Ask Amazon Q
main.tf
GNU nano 6.2
provider "aws" {
  region = var.region
}

# ----- VPC -----
resource "aws_vpc" "main_vpc" {
  cidr_block = "10.0.0.0/16"
}

# ----- Subnets -----
resource "aws_subnet" "public_subnet" {
  vpc_id         = aws_vpc.main_vpc.id
  cidr_block     = "10.0.1.0/24"
  availability_zone = "ap-south-1a"
  map_public_ip_on_launch = true
}

resource "aws_subnet" "private_subnet" {
  vpc_id         = aws_vpc.main_vpc.id
  cidr_block     = "10.0.2.0/24"
  availability_zone = "ap-south-1b"
}

# ----- Internet Gateway -----
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main_vpc.id
}

# ----- Route Table -----

```

- variables.tf – stores variable definitions

```

aws
Search [Alt+S] Ask Amazon Q
variables.tf
GNU nano 6.2
variable "region" {
  description = "AWS Region"
  default     = "ap-south-1"
}

variable "instance_type" {
  description = "EC2 instance type"
  default     = "t3.micro"
}

variable "db_username" {
  description = "Database username"
}

variable "db_password" {
  description = "Database password"
}

variable "bucket_name" {
  description = "S3 bucket name"
}

```

- terraform.tfvars – stores variable values

```

aws
Search [Alt+S] Ask Amazon Q
terraform.tfvars
GNU nano 6.2
region = "ap-south-1"

instance_type = "t3.micro"

db_username = "admin"

db_password = "Admin12345"

bucket_name = "my-terraform-3tier-bucket-lavanya123"

```

- outputs.tf – displays important output

```

aws
Search [Alt+S] Ask Amazon Q
output.tf
GNU nano 6.2
output "ec2_public_ip" {
  value = aws_instance.web_server.public_ip
}

output "vpc_id" {
  value = aws_vpc.main_vpc.id
}

output "rds_endpoint" {
  value = aws_db_instance.database.endpoint
}

output "s3_bucket_name" {
  value = aws_s3_bucket.bucket.bucket
}

```

STEP 5: Initialize Terraform

- Initialize the Terraform working directory.
- terraform init

```
ubuntu@ip-172-31-32-58:~/terraform-3tier$ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/aws from the dependency lock file
- Using previously-installed hashicorp/aws v6.35.1

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
ubuntu@ip-172-31-32-58:~/terraform-3tier$
```

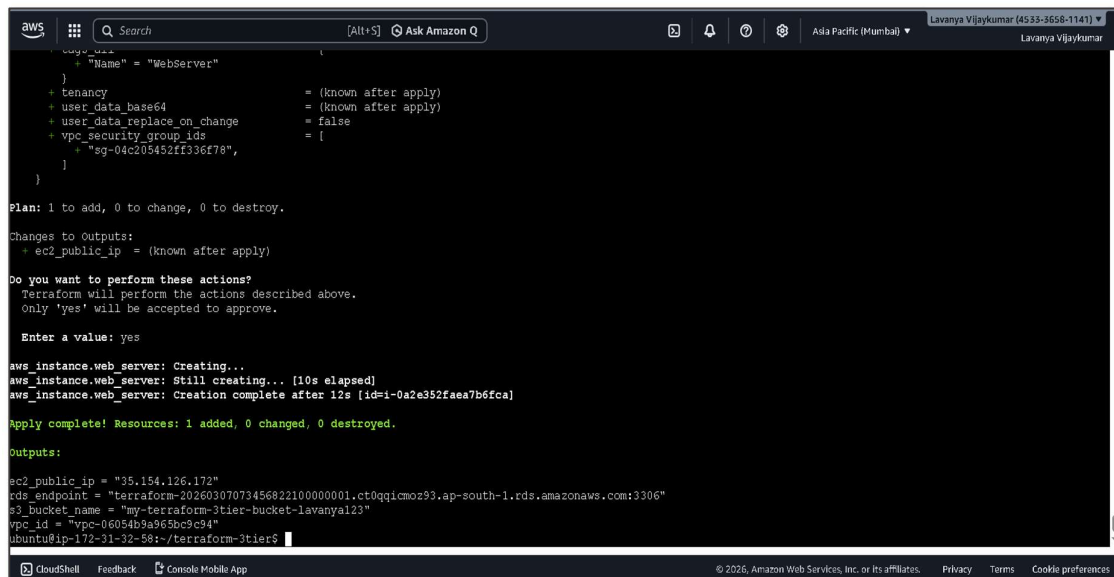
STEP 6: Format and Validate Terraform Configuration

- Check whether the Terraform files are correctly written.
- terraform validate ,terraform fmt

```
ubuntu@ip-172-31-32-58:~/terraform-3tier$ terraform fmt
ubuntu@ip-172-31-32-58:~/terraform-3tier$ terraform validate
Success! The configuration is valid.
```

STEP 7: Generate Execution Plan and Deploy Infrastructure

- Preview the infrastructure that Terraform will create, Apply the Terraform configuration to create AWS resources.
- terraform plan
- terraform apply
- Type yes when prompted.



```
aws
[+] Search [Alt+S] Ask Amazon Q
[+] Asda Pacific (Mumbai) Lavanya Vijaykumar (4533-9658-1141) Lavanya Vijaykumar

+ user_data_base64 = (known after apply)
+ user_data_replace_on_change = false
+ vpc_security_group_ids = [
  + "sg-04c205452ff336f78",
]
}

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ec2_public_ip = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

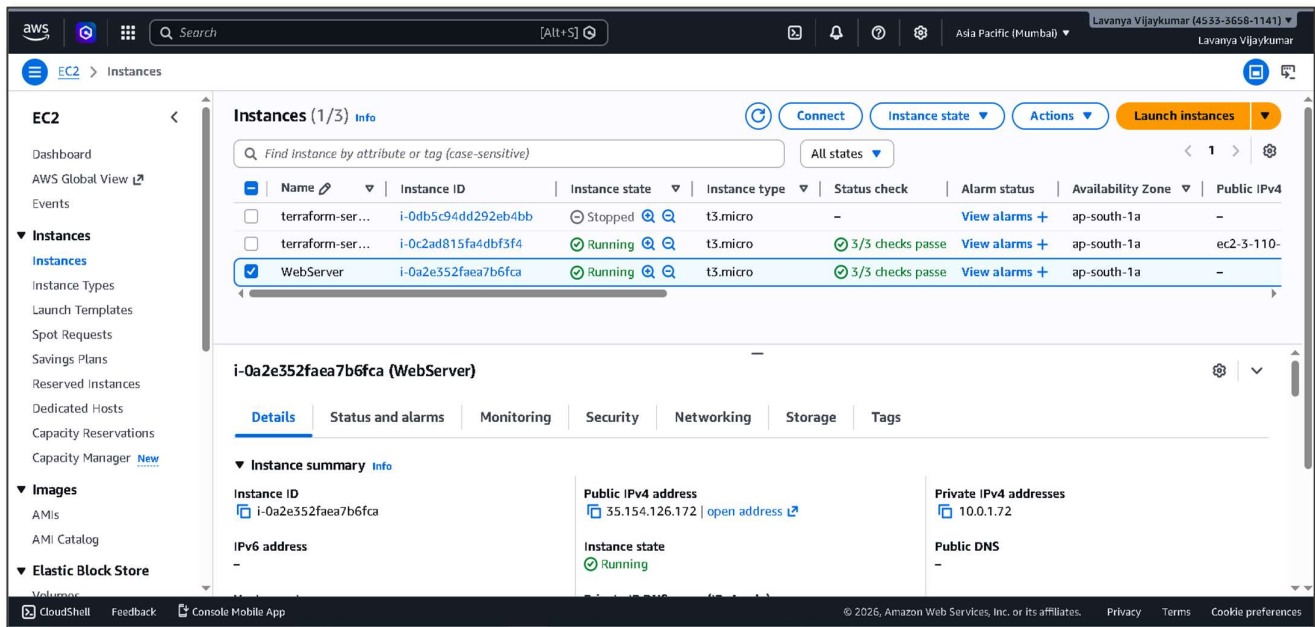
  Enter a value: yes

aws_instance.web_server: Creating...
aws_instance.web_server: Still creating... [10s elapsed]
aws_instance.web_server: Creation complete after 12s [id=i-0a2e352faea7b6fca]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:
ec2_public_ip = "35.154.126.172"
rds_endpoint = "terraform-20260307073456822100000001.ct0qqicmoz93.ap-south-1.rds.amazonaws.com:3306"
s3_bucket_name = "my-terraform-3tier-bucket-lavanya123"
vpc_id = "vpc-06054b9a965bc9c94"
ubuntu@ip-172-31-32-58:~/terraform-3tier$
```

STEP 8: Outputs



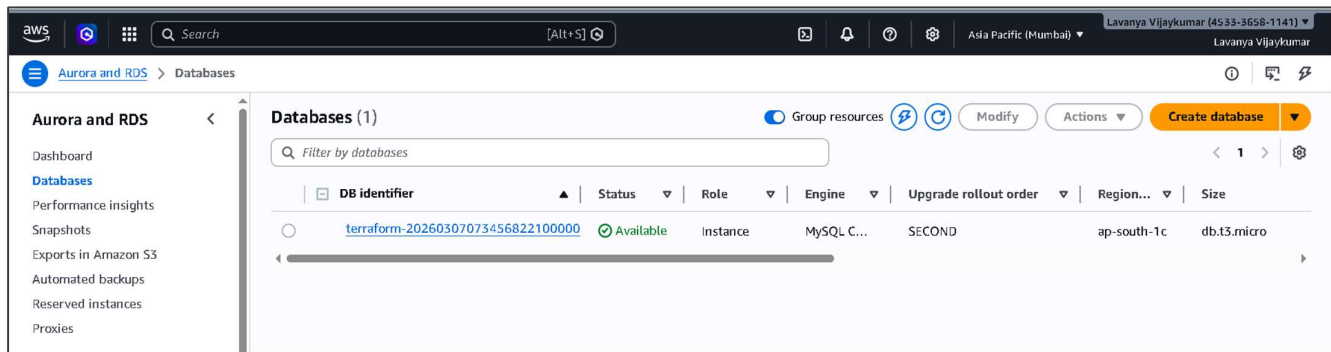
EC2 Instances (1/3) Info

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
terraform-ser...	i-0db5c94dd292eb4bb	Stopped	t3.micro	-	View alarms +	ap-south-1a	-
terraform-ser...	i-0c2ad815fa4dbf3f4	Running	t3.micro	3/3 checks passe	View alarms +	ap-south-1a	ec2-3-110-
WebServer	i-0a2e352faea7b6fca	Running	t3.micro	3/3 checks passe	View alarms +	ap-south-1a	-

i-0a2e352faea7b6fca (WebServer)

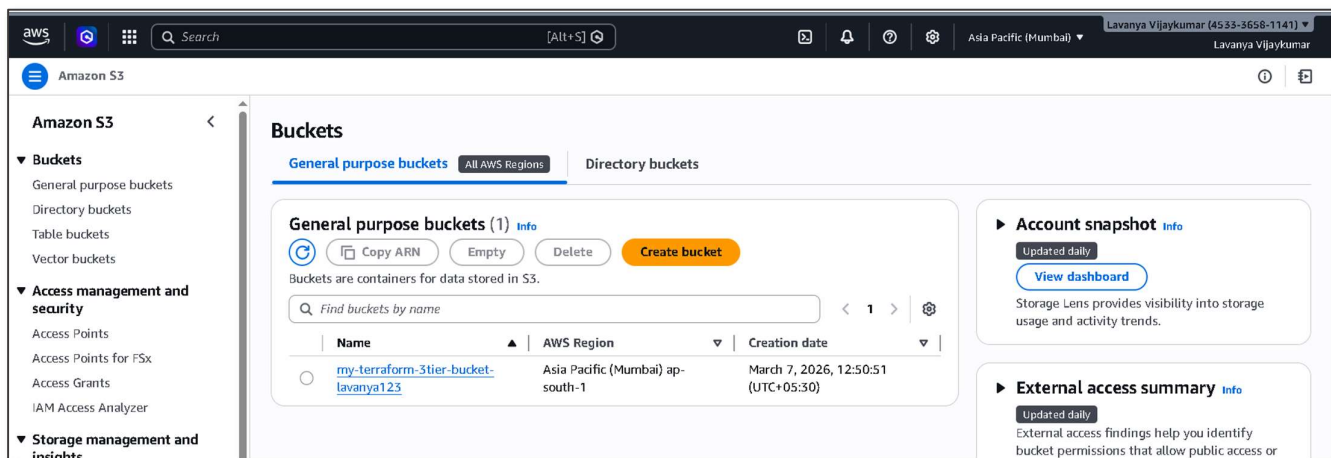
Instance summary

Instance ID i-0a2e352faea7b6fca	Public IPv4 address 35.154.126.172 open address	Private IPv4 addresses 10.0.1.72
IPv6 address -	Instance state Running	Public DNS -



Aurora and RDS Databases (1)

DB identifier	Status	Role	Engine	Upgrade rollout order	Region...	Size
terraform-20260307073456822100000	Available	Instance	MySQL C...	SECOND	ap-south-1c	db.t3.micro



Amazon S3 Buckets

General purpose buckets (1) Info

Buckets are containers for data stored in S3.

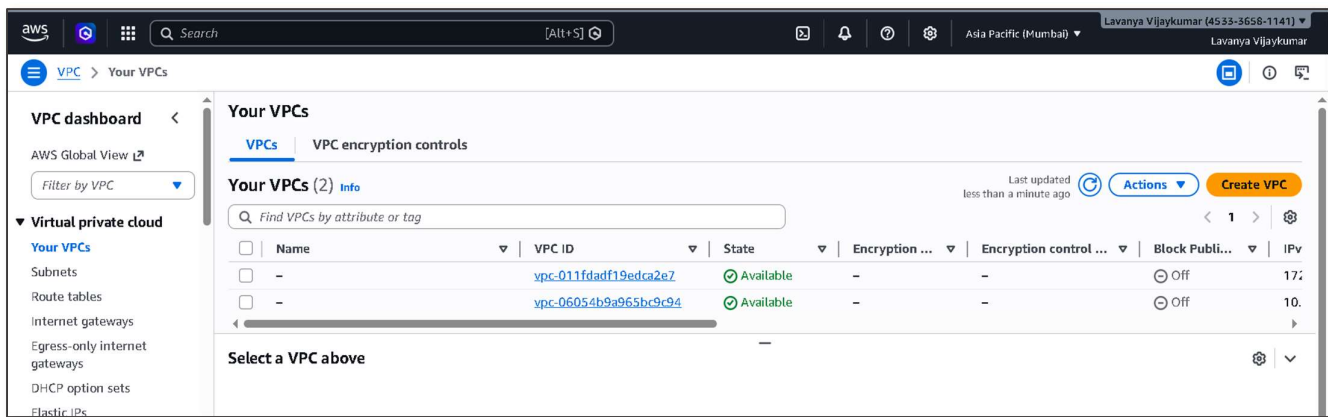
Name	AWS Region	Creation date
my-terraform-3tier-bucket-lavanya123	Asia Pacific (Mumbai) ap-south-1	March 7, 2026, 12:50:51 (UTC+05:30)

Account snapshot (Updated daily) | [View dashboard](#)

Storage Lens provides visibility into storage usage and activity trends.

External access summary (Updated daily) | [Info](#)

External access findings help you identify bucket permissions that allow public access or



Troubleshooting:

Issue 1: RDS DB Creation Error – Invalid Password

Error: InvalidParameterValue: MasterUserPassword is not valid

Cause: Password contains invalid characters such as /, @, ", or spaces.

Solution: Use a password with only alphanumeric characters and allowed symbols.

Conclusion:

This project demonstrates the design, deployment, and management of a secure 3-tier cloud architecture on Amazon Web Services using Terraform. By using Infrastructure as Code, the entire infrastructure—including VPC, public and private subnets, EC2 instances, RDS database, and S3 storage—was provisioned automatically, ensuring consistency, repeatability, and easy scalability. The project highlights best practices in cloud networking, resource isolation, and automation, reducing manual errors and simplifying infrastructure management. Overall, it showcases modern DevOps practices, efficient cloud deployment, and the practical application of Terraform to build robust, scalable, and maintainable cloud solutions.